

JAX 框架入门介绍

面向机器学习研究者与工程师

2026 年 8 月 7 日

摘要

JAX 是由 Google Research 开发的高性能数值计算框架，它将 NumPy 的简洁接口、自动微分的能力和硬件加速的性能融为一体。本文简要介绍 JAX 的核心设计理念、四大函数变换原语及其典型应用场景，帮助读者快速上手这一在机器学习研究领域日益流行的工具。

目录

1	什么是 JAX?	3
1.1	核心优势	3
2	安装与快速上手	3
2.1	安装	3
2.2	第一个例子	3
3	四大核心变换	4
3.1	grad — 自动微分	4
3.2	jit — 即时编译加速	5
3.3	vmap — 自动向量化	5
3.4	pmap — 并行映射	5
3.5	变换的组合	6
4	JAX 的纯函数式约束	6
4.1	常见的”坑”与解决方案	6
4.1.1	不可变数组	6
4.1.2	显式随机数管理	6
5	JAX 生态与上层库	7
6	JAX 与 PyTorch 的对比	7

7	完整示例：用 JAX 训练线性回归	7
8	学习资源	8
9	总结	9

1 什么是 JAX?

JAX 的全称可以理解为 **NumPy on Accelerators**。它的核心定位可以用一句话概括：

$$JAX = NumPy + \text{自动微分} + GPU/TPU \text{ 加速编译}$$

与 PyTorch、TensorFlow 等主流深度学习框架相比，JAX 的设计哲学有显著不同：它不是一个“大而全”的训练框架，而是一组**可组合的函数变换工具**。用户以纯函数的形式定义计算逻辑，JAX 则提供一套高阶函数来对这些计算进行变换——求导、向量化、编译加速、并行化——就像搭积木一样自由组合。

1.1 核心优势

- **NumPy 兼容**：`jax.numpy` 提供了与 NumPy 几乎一致的 API，迁移成本极低。
- **自动微分**：原生支持前向模式（JVP）和反向模式（VJP），可轻松计算高阶导数。
- **即时编译（JIT）**：通过 XLA 编译器将 Python 代码编译为高效的 GPU/TPU 内核。
- **向量化**：`vmap` 自动将单样本函数提升为批处理函数，无需手动编写循环。
- **纯函数式设计**：无副作用、不可变数据，使程序变换在语义上安全且可预测。

2 安装与快速上手

2.1 安装

JAX 的安装非常简单，可通过 `pip` 完成。以 CPU 版本为例：

```
1 pip install jax jaxlib
```

若需 GPU 支持，请参考官方文档安装对应的 CUDA 版本。

2.2 第一个例子

下面是一个最简单的 JAX 程序——计算函数 $f(x) = x^2$ 在 $x = 3.0$ 处的导数：

```
1 import jax
2 import jax.numpy as jnp
3
4 # 定义函数
5 def f(x):
```

```

6     return x ** 2
7
8     # 使用 jax.grad 自动求导
9     df = jax.grad(f)
10
11    # 计算  $f'(3.0) = 2 * 3.0 = 6.0$ 
12    print(df(3.0))    # 输出: 6.0

```

jax.grad 接受一个标量值函数，返回其梯度函数。这一设计体现了 JAX 的核心理念：函数变换——输入一个函数，输出另一个函数。

3 四大核心变换

JAX 的能力建立在四个核心变换之上，它们可以任意组合使用。

3.1 grad — 自动微分

grad 用于计算标量函数关于其参数的梯度，底层基于反向模式自动微分（即反向传播）。

```

1 def loss(w, b, x, y):
2     pred = w * x + b
3     return (pred - y) ** 2
4
5     # 分别对 w 和 b 求偏导
6     grad_w = jax.grad(loss, argnums=0)    # 第 0 个参数
7     grad_b = jax.grad(loss, argnums=1)    # 第 1 个参数
8
9     print(grad_w(2.0, 1.0, 3.0, 4.0))    #  $d(loss)/dw$ 
10    print(grad_b(2.0, 1.0, 3.0, 4.0))    #  $d(loss)/db$ 

```

高阶导数同样轻而易举——只需对 grad 反复应用：

```

1 f = lambda x: x ** 3
2 df = jax.grad(f)           # 一阶导:  $3x^2$ 
3 d2f = jax.grad(df)         # 二阶导:  $6x$ 
4 d3f = jax.grad(d2f)        # 三阶导:  $6$ 
5
6 print(d2f(2.0))            # 输出: 12.0

```

3.2 jit — 即时编译加速

jit (Just-In-Time compilation) 通过 XLA 编译器将函数编译为优化的机器码，消除 Python 解释器的开销：

```
1 @jax.jit
2 def heavy_computation(x):
3     for _ in range(1000):
4         x = x * 2
5     return x
6
7 # 首次调用时编译，后续调用直接执行编译后的代码
8 result = heavy_computation(jnp.array(1.0))
```

在实际的深度学习训练中，jit 可以将训练步骤的执行速度提升数倍至数十倍。

3.3 vmap — 自动向量化

vmap 将一个处理单个样本的函数自动提升为处理一批样本的函数，无需手动编写批处理逻辑：

```
1 # 单个样本的预测
2 def predict(params, x):
3     w, b = params
4     return jnp.dot(w, x) + b
5
6 # 自动向量化为批处理
7 batched_predict = jax.vmap(predict, in_axes=(None, 0))
8
9 # 对一批输入进行预测
10 batch_x = jnp.ones((32, 784)) # 32 个样本
11 results = batched_predict(params, batch_x) # 形状: (32,)
```

in_axes 参数指定每个输入沿哪个轴进行映射：None 表示不映射（广播），0 表示沿第 0 轴映射。

3.4 pmap — 并行映射

pmap 与 vmap 类似，但将计算分配到多个 GPU/TPU 设备上并行执行：

```
1 # 假设有 8 个 GPU
2 parallel_train = jax.pmap(train_step)
3
```

```

4 # 数据自动分配到 8 个设备上并行训练
5 results = parallel_train(batched_data)

```

3.5 变换的组合

JAX 最强大的特性在于这些变换可以自由嵌套。例如：

```

1 # 编译加速 + 向量化 + 求导
2 fast_batch_grad = jax.jit(jax.vmap(jax.grad(loss)))

```

这一行代码的含义是：对损失函数求梯度，然后向量化到一批数据上，最后编译加速。整个过程无需手动处理任何底层细节。

4 JAX 的纯函数式约束

JAX 的变换（jit, grad, vmap 等）要求被变换的函数是**纯函数**（pure function），即：

- 输出仅依赖于输入参数，不依赖外部全局状态。
- 不产生副作用（如修改全局变量、打印输出等）。

4.1 常见的”坑”与解决方案

4.1.1 不可变数组

JAX 数组（jnp.ndarray）是不可变的：

```

1 x = jnp.array([1, 2, 3])
2 # x[0] = 10 # 错误！JAX 数组不支持原地修改
3
4 # 正确做法：使用 .at 语法
5 x = x.at[0].set(10)
6 print(x) # [10, 2, 3]

```

4.1.2 显式随机数管理

JAX 不使用全局随机状态，而是要求显式传递 PRNG key：

```

1 key = jax.random.PRNGKey(42)
2
3 # 每次使用需要拆分 key
4 key, subkey = jax.random.split(key)
5 sample = jax.random.normal(subkey, shape=(3,))

```

```

6
7 key, subkey = jax.random.split(key)
8 another_sample = jax.random.normal(subkey, shape=(3,))

```

这种设计虽然略显繁琐，但确保了随机数的可复现性和并行安全性。

5 JAX 生态与上层库

JAX 本身是一个底层计算框架，社区在此基础上构建了丰富的上层库：

表 1: JAX 生态系统中的主要上层库

库名	用途	类比
Flax	神经网络模块化	PyTorch nn.Module
Haiku	神经网络模块化（DeepMind）	PyTorch nn.Module
Optax	优化器与梯度变换	torch.optim
Optax	优化器与梯度变换	torch.optim
Equinox	基于 PyTree 的模块化	nn.Module + JAX
Diffrax	可微分 ODE/SDE 求解器	torchdiffeq
Orbax	检查点与模型序列化	torch.save/load

6 JAX 与 PyTorch 的对比

表 2: JAX 与 PyTorch 的核心差异

维度	JAX	PyTorch
编程范式	纯函数式	面向对象 + 命令式
数组可变性	不可变	可变（原地操作）
编译加速	内置 JIT（XLA）	torch.compile（较新）
向量化	vmap（自动）	手动编写批处理逻辑
自动微分	前向 + 反向 + 高阶	主要为反向模式
随机数管理	显式 PRNG key	隐式全局状态
学习曲线	较陡（函数式思维）	较平缓（命令式直觉）

7 完整示例：用 JAX 训练线性回归

下面是一个完整的线性回归训练示例，展示了 JAX 四大变换的综合运用：

```

1 import jax
2 import jax.numpy as jnp
3
4 # 1. 生成合成数据
5 key = jax.random.PRNGKey(0)
6 key, subkey = jax.random.split(key)
7 X = jax.random.normal(subkey, (100, 3))
8 w_true = jnp.array([1.0, 2.0, -3.0])
9 y = X @ w_true + 0.1 * jax.random.normal(
10     jax.random.split(key)[1], (100,))
11
12 # 2. 定义模型与损失函数
13 def predict(params, x):
14     return jnp.dot(x, params)
15
16 def loss(params, x, y):
17     pred = predict(params, x)
18     return jnp.mean((pred - y) ** 2)
19
20 # 3. 定义训练步骤 (jit 编译加速)
21 @jax.jit
22 def train_step(params, x, y, lr=0.01):
23     grads = jax.grad(loss)(params, x, y)
24     return params - lr * grads
25
26 # 4. 训练循环
27 params = jnp.zeros(3)
28 for epoch in range(100):
29     params = train_step(params, X, y)
30     if epoch % 20 == 0:
31         print(f"Epoch {epoch}: loss = {loss(params, X, y):.4f}")
32
33 print(f"Learned weights: {params}")
34 # 期望输出接近 [1.0, 2.0, -3.0]

```

8 学习资源

- 官方文档: <https://jax.readthedocs.io>

- **JAX 101 教程：**官方提供的从零开始的系列教程
- **JAX in Action：**Manning 出版社的实战书籍
- **DeepMind 的 JAX 生态：**
<https://www.deepmind.com/blog/using-jax-to-accelerate-our-research>

9 总结

JAX 以函数变换为核心抽象，将自动微分、编译加速和并行计算统一在一个简洁而强大的框架中。虽然其纯函数式设计范式需要一定的学习成本，但一旦掌握，便能以极高的表达力和组合性来构建复杂的机器学习研究流水线。

对于追求计算效率、需要高阶微分或希望探索新型模型架构的研究者而言，JAX 是一个值得深入学习的工具。